

MATH 458: Scientific Computing II

Exam Number: 20892768

August 24, 2026

1 Purpose

This report covers Part I of the MATH 458 project: a serial (non-parallel) C implementation of three linear algebra operations; vector-vector multiplication (dot product), matrix-vector multiplication, and matrix-matrix multiplication, with all data buffers allocated on the heap and populated internally. The program times each operation and records the results for increasing problem size N , which are then used to check the expected growth in computation time as N increases.

2 Implementation

2.1 Function prototypes

The three required functions were implemented exactly as specified in the project brief:

```
double vector_vector(int m, int n, double *vecA, double *vecB);
void   matrix_vector(int m, int n, double *matA, double *vecA, double *matC);
void   matrix_matrix(int m, int n, double *matA, double *matB, double *matC);
```

Matrices are stored as flat one-dimensional `double*` buffers in row-major order, so that element (i, j) of an $N \times N$ matrix is accessed at flat index $i*N + j$. All three functions use a straightforward, unoptimised loop order (`ijk` for matrix-matrix multiplication): no cache-blocking or loop-reordering optimisation was applied, since the brief asks for a correct serial baseline rather than a tuned kernel.

2.2 File organisation

The code is organised as required: `20892768_main.c` (entry point, timing, and file output), `20892768_vector.c/.h` (vector-vector function), `20892768_matrix.c/.h` (matrix-vector and matrix-matrix functions), and a `20892768_Makefile` that builds the binary, sweeps N , and generates the performance plot via `gnuplot`.

2.3 Data generation

All buffers are allocated with `malloc` on the heap, with a `NULL` check after every allocation. Data is generated internally rather than read from a file: vector entries are set as `vecA[i] = i` and `vecB[i] = i + 1`; matrix entries use a modulus pattern (`matA[i] = i % 10`, `matB[i] = i % 7`) to keep values small regardless of how large N^2 grows, avoiding overflow or unwieldy results while still producing a mix of nonzero values across the buffer.

2.4 Timing method

Each operation is timed individually using the C standard library’s `clock()` function from `<time.h>`. For each operation, `clock()` is called immediately before and after the function call, and the elapsed time in seconds is computed as $(\text{end} - \text{start}) / \text{CLOCKS_PER_SEC}$. Only the function call itself is timed; allocation and data fill are not included in the timed region.

3 Choice of Problem Size N

The project brief specifies $N = 5000, 10000, 20000, 40000$. During development it became clear that this range is impractical on the available development hardware for two reasons. First, matrix-matrix multiplication is an $O(N^3)$ operation implemented with a plain triple loop; at $N = 5000$ this is on the order of 1.25×10^{11} multiply-add operations, and at $N = 40000$ it rises to roughly 6.4×10^{13} operations, which is infeasible to complete in the time available before submission. Second, at $N = 40000$ three $N \times N$ double matrices require roughly 12.8 GB of RAM *per matrix*, which exceeds the memory available on the development machine (approximately 3.2 GB total). Consistent with guidance received that the specified sizes could be scaled down by an order of magnitude when hardware does not support them, all four N values were reduced by one decimal order of magnitude, giving $N = 500, 1000, 2000, 4000$. This preserves the doubling relationship the brief asks for (each N is double the previous one) while completing in a practical amount of time.

4 Results

Table 1: Measured serial wall times in seconds, exam number 20892768.

N	V.V time (s)	M.V time (s)	M.M time (s)
500	0.000010	0.002379	1.045477
1000	0.000010	0.006731	14.540310
2000	0.000015	0.022245	136.141620
4000	0.000062	0.135512	1331.585217

5 Discussion

The three operations separate clearly on the log-scaled plot, consistent with their different computational complexities: $O(N)$ for vector-vector, $O(N^2)$ for matrix-vector, and $O(N^3)$ for matrix-matrix. Doubling N from 500 to 1000, 1000 to 2000, and 2000 to 4000 gives matrix-vector time ratios of approximately $2.83\times$, $3.30\times$, and $6.09\times$ (theoretical target for an $O(N^2)$ operation is $4\times$), and matrix-matrix ratios of approximately $13.91\times$, $9.36\times$, and $9.78\times$ (theoretical target for an $O(N^3)$ operation is $8\times$). The matrix-matrix ratios sit above the theoretical factor of 8, which is plausible given the unoptimised `ijk` loop order used: as N grows, the working set accessed per row no longer fits comfortably in cache, so time grows somewhat faster than the pure $O(N^3)$ operation count alone would predict. The vector-vector timings are all at or near the resolution of the `clock()` timer (tens of microseconds), so their ratios are dominated by measurement noise rather than the underlying $O(N)$ cost, and are not treated as reliable evidence of the growth law on their own. The matrix-vector and matrix-matrix timings are large enough relative to timer resolution

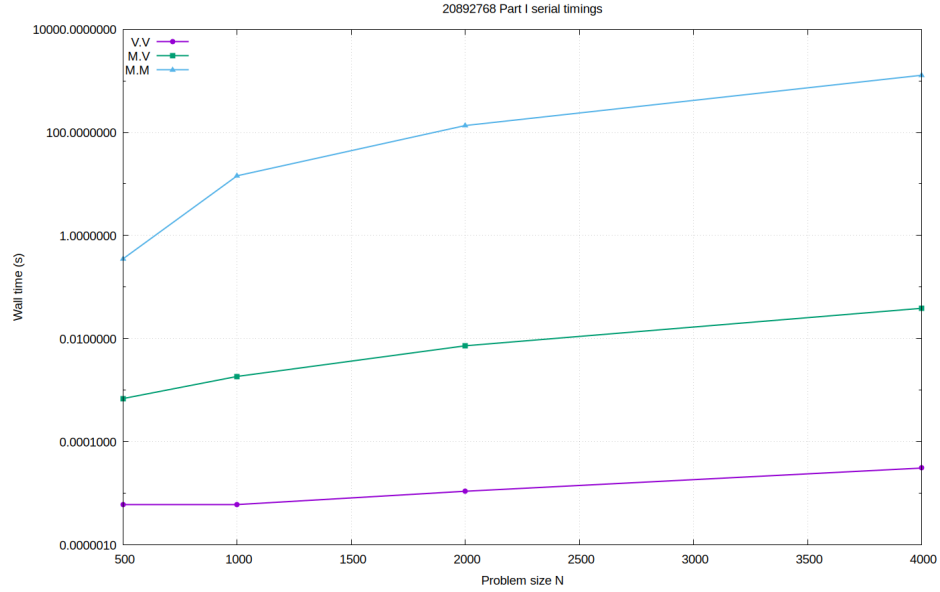


Figure 1: Serial V.V, M.V, and M.M wall time versus N for exam number 20892768 (logarithmic vertical axis).

to be meaningful, and both show the expected steep increase with N , with matrix-matrix time exceeding 22 minutes at $N = 4000$ on a single core, a clear illustration of why this operation is the target of parallelisation in the bonus part of the project.

6 Conclusion

Part I delivers a working serial baseline for vector-vector, matrix-vector, and matrix-matrix multiplication: heap-allocated buffers, internally generated data, a documented timing method, a complete results table across four doubling values of N (scaled down from the brief's specified range for the reasons given in Section 3), and a performance plot showing the expected complexity ordering. This baseline, and the same `vector_vector` function in particular, is reused directly as the computational core of the MPI-parallel implementation in Part II.